

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	Position Swapper	Documentation quality	High	
Timeline	2026-01-14 through 2026-01-19	Test quality	Medium	
Language	Solidity	Total Findings	1	Fixed: 1
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review	High severity findings ⓘ	0	
Specification	Audit Scope - Position Swapper ⓘ	Medium severity findings ⓘ	0	
Source Code	https://github.com/VenusProtocol/venus-periphery ⓘ #88f8d71 ⓘ	Low severity findings ⓘ	1	Fixed: 1
Auditors	- Ibrahim Abouzied Auditing Engineer - Jennifer Wu Auditing Engineer - Mostafa Yassin Auditing Engineer	Undetermined severity findings ⓘ	0	
		Informational findings ⓘ	0	

Summary of Findings

This audit covered a contract upgrade for the Position Swapper. The Position Swapper is a smart contract that lets users adjust their existing Venus positions—whether collateral or debt—in a single atomic transaction. Venus Flash Loans are used as the source of temporary liquidity, such that the user does not need to pre-fund the swap from their own wallet.

The audit uncovered an issue where the `initiator` is not checked in the Flashloan Callback (**VENUS-1**). While this does not cause issues in the current implementation, it could pose a security threat if the flashloan surface expands in future upgrades.

Fix-Review Update: The Venus team has resolved the issue and implemented or acknowledged all of our suggestions.

ID	DESCRIPTION	SEVERITY	STATUS
VENUS-1	Missing Initiator Validation in Flashloan Callback	• Low ⓘ	Fixed

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.



Disclaimer

Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence

- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
 1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
 2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
 3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
 1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
 2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

Files Included

- contracts/PositionSwapper/IPositionSwapper.sol
- contracts/PositionSwapper/PositionSwapper.sol

Key Actors And Their Capabilities

The contract owner can call:

- `sweepToken(IERC20Upgradeable token)` : Sends the contract's full token balance to the owner. Intended for reclaiming dust.
- `sweepNative()` : Sends the contract's full native token balance to the owner. Intended for reclaiming dust.

The `COMPTROLLER` can call:

- `executeOperation()` : This is a callback function for executing the flashloan and acts as a router for the user's desired swap.

Findings

VENUS-1 Missing Initiator Validation in Flashloan Callback

Low ⓘ Fixed



Update

Marked as "Fixed" by the client.
Addressed in: `9a3e27cff5ad2a5791edecf975adb4a9ef734cd7` .

File(s) affected: `PositionSwapper.sol`

Description: `PositionSwapper.executeOperation` validates `msg.sender == COMPTROLLER` and checks array lengths, but does not validate the flashloan `initiator` . The callback accepts any `initiator` as long as the call originates from the `Comptroller` .

Although the `Comptroller` flashloan enforces `msg.sender == onBehalf` or a pre-approved delegate via `approvedDelegates[onBehalf][msg.sender]` , the `PositionSwapper.executeOperation` function does not ensure the callback was initiated by `PositionSwapper` itself.

In the current system, `Comptroller` flashloans are gated so that reentrant callbacks through `SwapHelper` are not possible. However, if a future upgrade introduces another contract that can invoke flashloans, a malicious contract could trigger `executeOperation` with attacker-controlled parameters against users who have pre-approved `PositionSwapper` and that new flashloaning contract.

This creates a forward-looking risk of swapping user positions without their intent if the flashloan surface expands in future upgrades.

Validating `initiator` to be `PositionSwapper` itself narrows the trust boundary of the callback and prevents unexpected behavior if the flashloan surface expands in future upgrades.

Exploit Scenario:

- 1. A future upgrade deploys `FlashloanRouter` that can call `Comptroller.flashLoan` on behalf of users.
- 2. A victim pre-approves `PositionSwapper` and also approves `FlashloanRouter` (e.g., for convenience or bundled workflows).
- 3. The attacker calls `FlashloanRouter.flashLoan(...)` and passes `onBehalf = victim` plus swap parameters that route through `receiver = PositionSwapper`.
- 4. In the same transaction, `FlashloanRouter` triggers a second flashloan before the transient state (e.g., `operationType`, `transientMarketFrom/To`) is cleared.
- 5. Comptroller calls `PositionSwapper.executeOperation`.
- 6. Because `initiator` is not validated, the callback proceeds and executes swaps using the victim's allowances, resulting in a swap the victim did not initiate.

Recommendation: Validate that the flashloan `initiator` equals the `PositionSwapper` contract address so the callback can only execute in the context of an internally initiated flashloan.

Auditor Suggestions

S1 No Validation that `marketFrom` and `marketTo` Are Different

Fixed

✓

Update
Marked as "Fixed" by the client.
Addressed in: `b3b5517c8e1f7c1b60b4bbef53af6fcecfa7f2c9`.

File(s) affected: `contracts/PositionSwapper/PositionSwapper.sol`

Description: The `swapCollateral` and `swapDebt` functions do not validate that the source and destination markets are different. A user could accidentally or intentionally swap from a market to itself, wasting gas and flash loan fees without achieving any position change.

Recommendation: Ensure that both markets are different.

S2 Return Data From `SWAP_HELPER.call()` Is Ignored

Fixed

✓

Update
Marked as "Fixed" by the client.
Addressed in: `063c9f9fe8db6c97014873d468e4199cfa433f96`.

File(s) affected: `contracts/PositionSwapper/PositionSwapper.sol`

Description: In `_performSwap`, the return data from the external call to `SwapHelper` is completely ignored. Only the success boolean is checked. This makes debugging failed swaps difficult and could mask important error information.

Recommendation: Consider handling the return value.

S3 Native Swap Functions Are Not Generic

Acknowledged

i

Update
Marked as "Acknowledged" by the client.
Addressed in: `5f3a8d83191daf1f776781f3d355189e10877800`.
The client provided the following explanation:

Acknowledged. These helpers are internal-only and are exclusively used for native ↔ wrapped-native flows; external entrypoints do not accept arbitrary token or market inputs.
To make this intent explicit and prevent future misuse, we have tighten the NatSpec to clearly document that `marketFrom` and `marketTo` must correspond to `NATIVE_MARKET` and `WRAPPED_NATIVE_MARKET`, respectively

File(s) affected: `contracts/PositionSwapper/PositionSwapper.sol`

Description: `_swapCollateralNativeToWrapped()` and `_swapDebtNativeToWrapped` accept `marketFrom` and `marketTo` . However, the function then constructs the flash-loan request using the hardcoded `WRAPPED_NATIVE_MARKET` and does not use `marketTo` .

This creates friction for readability/maintainability. The signature suggests the function is generic, but the actual behavior is implicitly fixed to `WRAPPED_NATIVE_MARKET` .

In future refactors or reuse, a developer might pass different `marketTo` values expecting them to control the borrowed market, while the contract continues to borrow from the hardcoded market.

Recommendation: Make the function’s intent explicit to prevent misuse. If the function is intended to be strictly for native and wrapped tokens, consider removing `marketFrom` / `marketTo` parameters and use `NATIVE_MARKET` / `WRAPPED_NATIVE_MARKET` directly.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Test Suite Results

The test suite was ran with the command `yarn test tests/hardhat/PositionSwapper/PositionSwapper.ts` .

```
positionSwapper
  swapCollateralNativeToWrapped
    ✓ should swapCollateralNativeToWrapped from vBNB to vWBNB (46ms)
  swapDebtNativeToWrapped
    ✓ should swapDebtNativeToWrapped from vBNB to vWBNB (126ms)
  swapCollateral
    ✓ should swapFullCollateral from vWBNB to vUSDT (53ms)
    ✓ should swapCollateralWithAmount from vWBNB to vUSDT (partial) and verify both balances (54ms)
  swapDebt
    ✓ should swapFullDebt from vWBNB to vUSDT (63ms)
    ✓ should swapDebtWithAmount from vUSDT to vBUSD (partial) (63ms)
  with flash loan fee
    ✓ should swapCollateralNativeToWrapped accounting for fee (43ms)
    ✓ should swapFullDebt with fee and keep target debt equal to requested amount (68ms)
  quoteFlashLoanAmount
    ✓ should net the required amount after subtracting fee from quoted borrow
    ✓ should net the required amount after subtracting fee from quoted borrow
    ✓ should net the required amount after subtracting fee from quoted borrow
    ✓ should net the required amount after subtracting fee from quoted borrow
    ✓ should net the required amount after subtracting fee from quoted borrow
    ✓ should net the required amount after subtracting fee from quoted borrow
    ✓ should net the required amount after subtracting fee from quoted borrow
  access and validation
    ✓ should revert swapCollateralWithAmount when amount is zero
    ✓ should revert swapDebtWithAmount when amount is zero
    ✓ should revert swapFullCollateral when no collateral balance
    ✓ should revert swapFullDebt when no borrow balance
    ✓ owner can sweep token; non-owner reverts
    ✓ owner can sweep native; non-owner reverts (70ms)
    ✓ should revert with MarketNotListed when market is not supported
    ✓ should revert with UnauthorizedCaller when caller is not user nor approved delegate
```

Code Coverage

Code coverage was gathered by running the command `npm run hardhat coverage --testfiles "tests/hardhat/PositionSwapper/PositionSwapper.ts"`.

`PositionSwapper` lacks `SwapperHelper` integration tests and currently relies on the `sweep` function, which provides insufficient coverage with integration with `SwapperHelper`. We encourage the team to add integration tests.

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Lines
contracts/PositionSwapper/	97.39	56.78	100	94.06	
IPositionSwapper.sol	100	100	100	100	
PositionSwapper.sol	97.39	56.78	100	94.06	... 750,781,782

Changelog

- 2026-01-19 - Initial report
- 2026-01-28 - Final report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 1100 audits and secured over \$200 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Polygon, Avalanche, Curve, Solana, Compound, Lido, MakerDAO, Arbitrum, OpenSea and the World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Solana, Polygon, TON, Cardano, Binance Smart Chain, Avalanche, Arbitrum, Flow
- DeFi: Lido, Ethena, Compound, Curve, Venus, PancakeSwap, Polymarket
- Infra/Staking: TrustWallet, Alchemy, Liquid Collective, Kiln, Galxe, API3, ssv.network, Luganodes
- Immersive: Virtuals Protocol, Wayfinder, OpenSea, Square Enix, Parallel, Camp, Decentraland
- Institutional: Visa, Circle, Revolut, Anthropic, OpenAI, Canton, Republicc, Arkham, Sequoia

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

